How Google Ads Was Able to Support 4.77 Billion Users With a SQL Database 🔥

#60: Break Into Google Spanner Architecture (5 Minutes)

NEO KIM

NOV 09, 2024 • PAID

Get my system design playbook for FREE on newsletter signup:

Subscribed

This post outlines Google Spanner architecture. You will find references at the bottom of this page if you want to go deeper.

- *Share this post & I'll send you some rewards for the referrals.*

Note: This post is based on my research and may differ from real-world implementation.

Once upon a time, 2 students decided to sell their university project for a million dollars.

Yet they failed to make the sale.

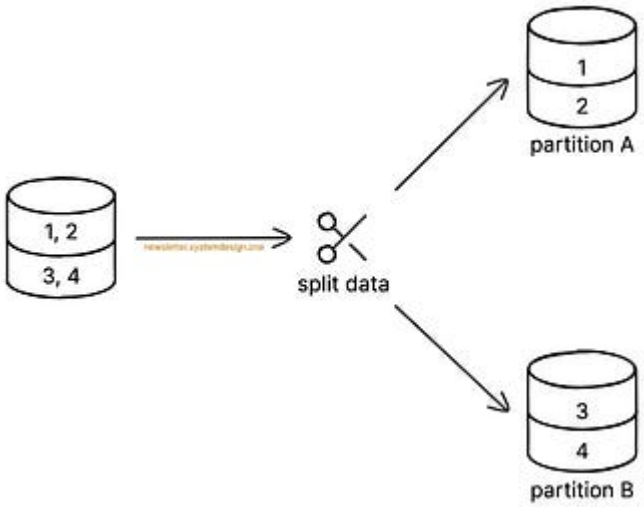
So they decided to develop it further and named it Google.

The main source of their income was from advertisement slots (**Ads**).

And their growth rate was explosive.

Yet they stored Ads data in MySQL for simplicity and reliability.

As more users joined, they partitioned MySQL for scale.



Key-Range Partitioning of MySQL to Scale

Although partitioning temporarily solved their scalability issues, there were newer problems.

Here are some of them:

1. Scalability

Their storage needs skyrocketed with growth.

And re-partitioning MySQL takes time as data must be moved between servers.

Yet they had extremely minimal downtime requirements.

So it became hard.

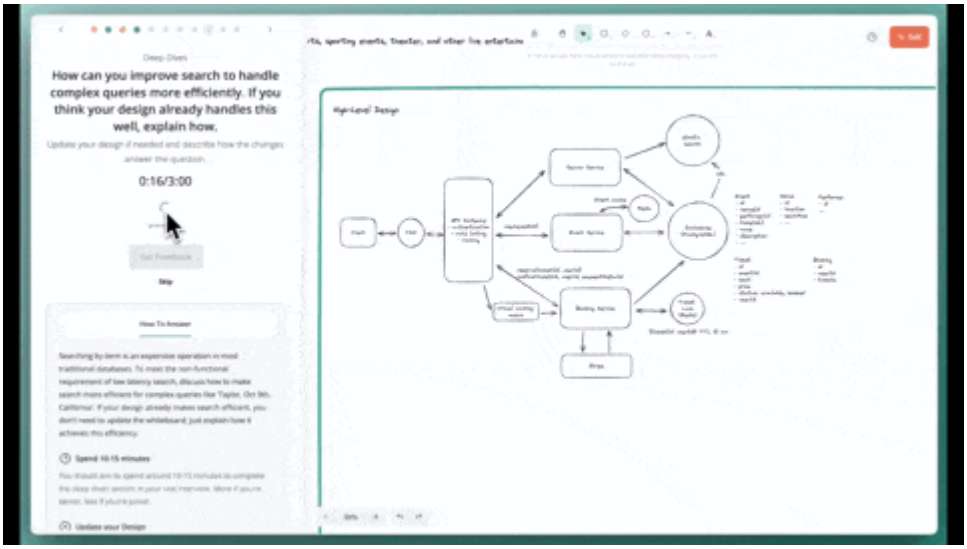
2. Transactions

They need ACID compliance for Ads data.

But transactions became difficult after partitioning.

Think of a **transaction** as a series of writes and reads.

System Design Guided Practice - Sponsor



Cloud Spanner Database

They need massive scalability of NoSQL.

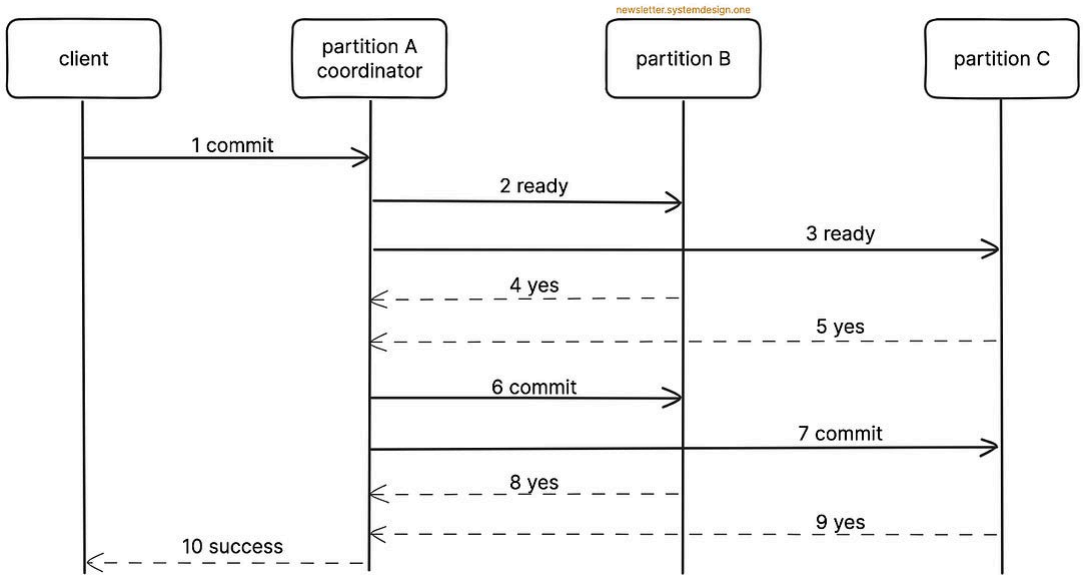
And ACID properties of MySQL.

So they created Spanner - *a distributed SQL database*.

Here's how:

1. Atomicity

Atomicity means a transaction is all or nothing. Put simply, a transaction must update data in different partitions at once.



Two-Phase Commit for Atomic Transactions

Yet it's difficult to ensure every partition will commit.

So they use the two-phase commit (**2PC**) protocol.

Here's how it works:

- Prepare phase: coordinator asks relevant partitions if they're ready to commit
- Commit phase: coordinator tells relevant partitions to commit if everyone agreed

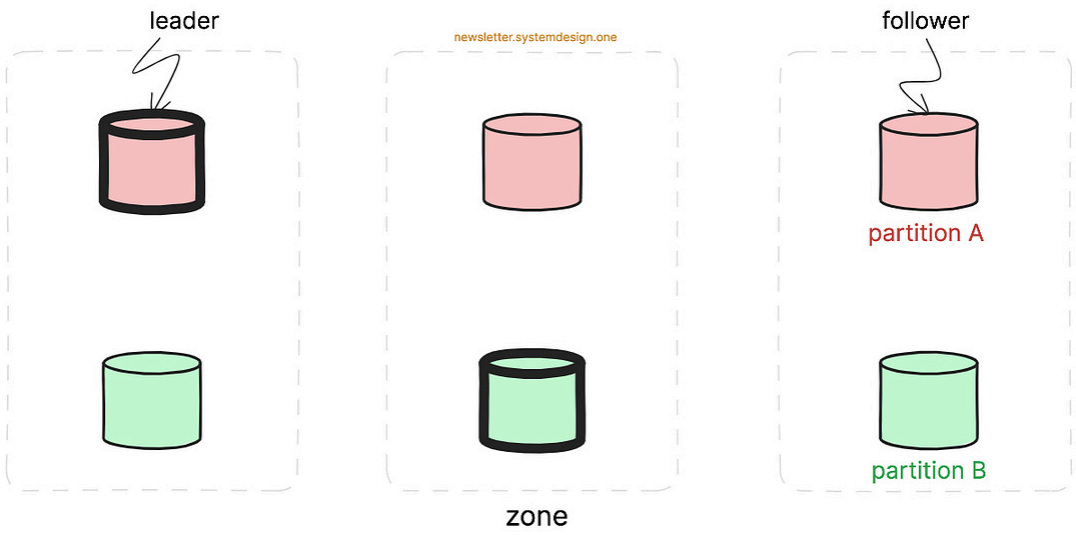
The transaction gets aborted if any partition isn't ready. Thus achieving atomicity.

Ready for the best part?

2. Consistency

They provide *strong consistency* at a global level.

This means if data gets updated in Europe, the same data would be shown in Asia. Put simply, a read will always return the latest write.



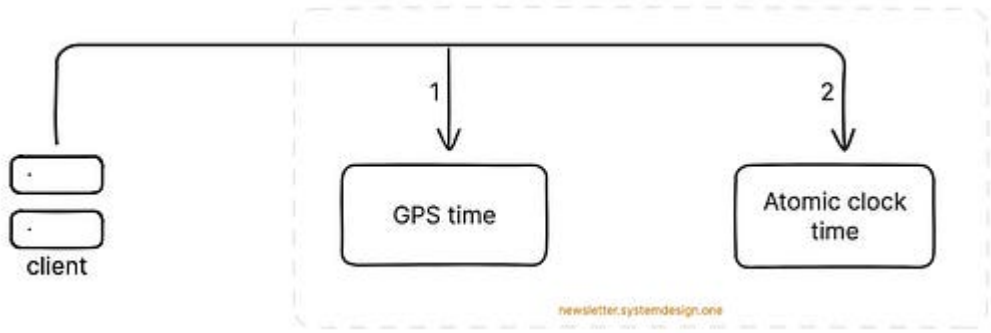
Deploying Partitions Across Zones

A database partition gets replicated across different zones for scalability. Think of a **zone** as a geographical location.

Yet it's important to coordinate writes among replicas to avoid data conflicts.

So they use the Paxos algorithm to find a partition leader. A partition **leader** is responsible for managing writes, while followers handle reads. Also different partition leaders might get deployed in separate zones.

Imagine **Paxos** as a technique to ensure consensus across distributed systems.



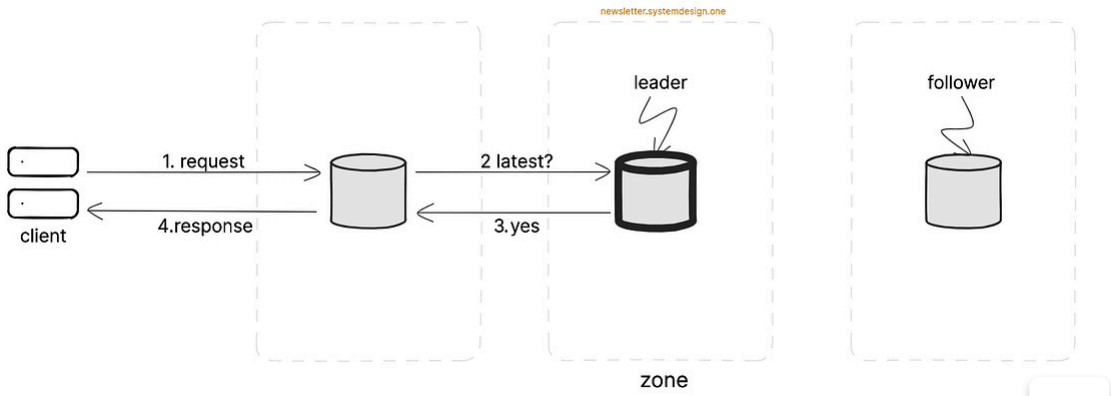
TrueTime Architecture

A simple way to achieve strong consistency is by ordering writes with timestamps. It allows a consistent view of data across servers.

Yet it's difficult to maintain the same time across every server in the world.

So they use TrueTime. Think of **TrueTime** as a combination of GPS receivers and atomic clocks. It finds the current time in each data center with high accuracy.

And each server synchronizes its quartz clock with TrueTime every 30 seconds.



Comparing Timestamps for Strong Consistency During Reads

Here's how they perform reads:

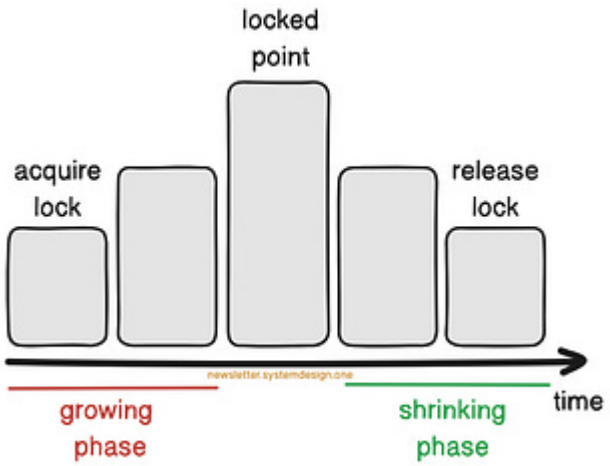
1. The request gets routed to the nearest zone even if it has a follower
2. The follower asks the leader for the latest timestamp of the requested data
3. The follower compares the leader's response with its timestamp
4. The follower responds to the client

Also the follower will wait for the leader to synchronize in case its data is outdated.

Ready for the next technique?

3. Isolation

Isolation means transactions don't interfere with each other.



Two-Phase Locking for Data Isolation During Writes

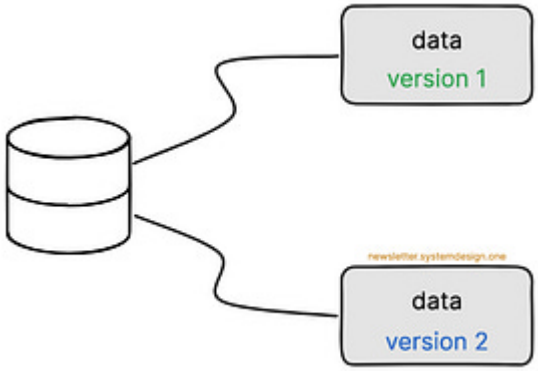
Yet it's difficult to avoid data conflicts due to concurrent transactions.

So they use two-phase locking (**2PL**) for data isolation during writes.

Here's how it works:

- Growing phase: transaction performs reads or writes once it acquires locks on data
- Shrinking phase: transaction releases locks one at a time after it's complete

Thus avoiding data conflicts on writes.



Snapshot Isolation During Reads

Besides they use snapshot isolation.

It's a multi-version concurrency control (**MVCC**) technique for *lock-free reads*. Think of **snapshot isolation** as a way to look at the database from a point in time.

It returns a specific version of data during reads and doesn't affect ongoing writes. Thus providing data isolation.

Here's how it works:

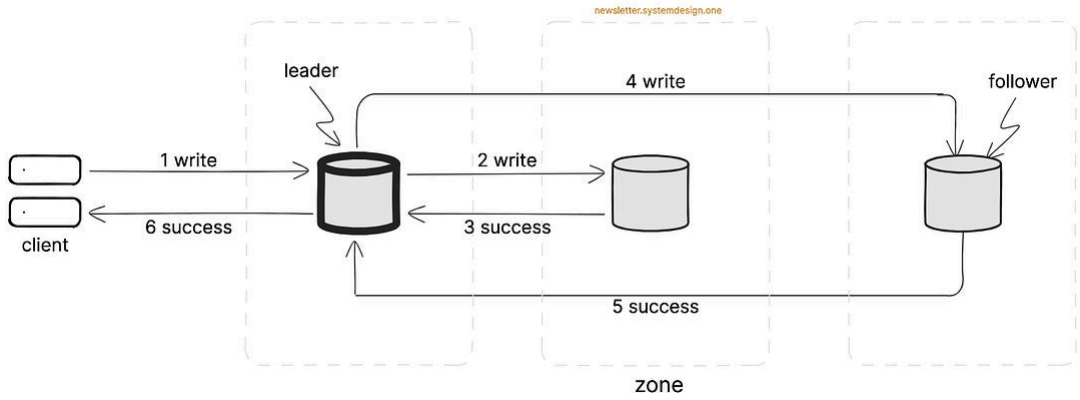
- The previous data isn't overwritten
- Instead a new value gets written along with a TrueTime timestamp

Also older versions get automatically removed after a specific period to save storage.

Onward.

4. Durability

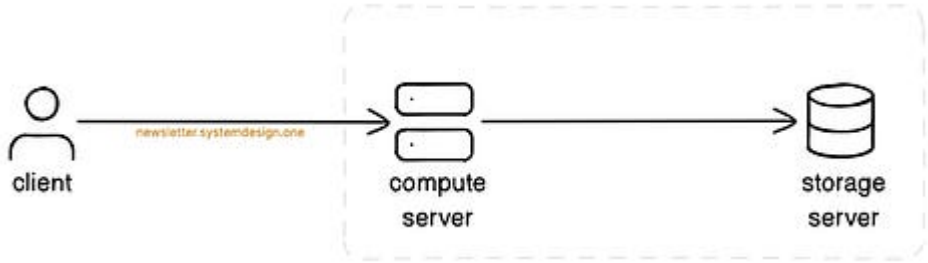
Durability means the result of a completed transaction never gets lost.



Synchronous Writes Using Paxos for Durability

They perform *synchronous writes* using Paxos. It gives durability as data is written to a majority of followers.

While data is then replicated to other followers based on the leader-follower replication technique.



High-Level Architecture of the Database Server

Besides they separate the compute and storage layers in the database server.

The compute layer handles reads and writes. While Google Colossus is used as the storage layer. Think of **Colossus** as a distributed file system for high performance and durability.

Spanner offers 99.999% availability.

While Google generated a whopping 237 billion dollars from Ads in 2023.

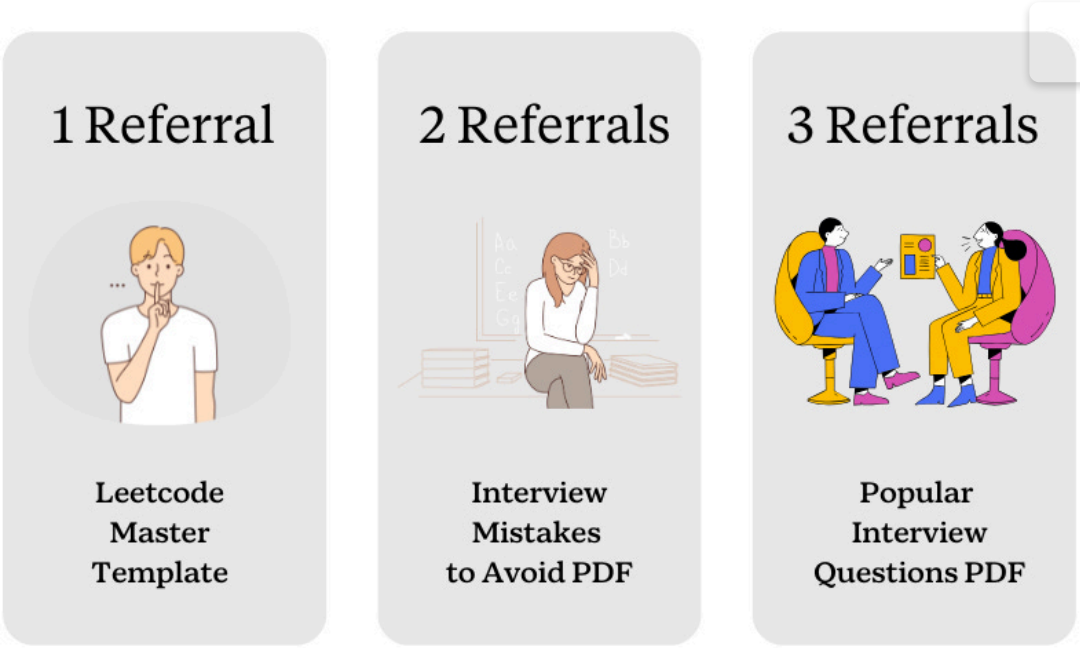
And became one of the most valuable companies in the world.

Subscribe to get simplified case studies delivered straight to your inbox:

Subscribed

Follow me on LinkedIn | YouTube | Threads | Twitter | Instagram

Thank you for supporting this newsletter. Consider sharing this post with your friends and get rewards. Y'all are the best.



Share

How Amazon S3 Works ✨

NEO KIM · OCTOBER 25, 2024

Read full story

How Facebook Was Able to Support a Billion Users via Software Load Balancer ⚡

NEO KIM · OCTOBER 11, 2024

Read full story

References

- Spanner: Google’s Globally Distributed Database White Paper
- F1: A Distributed SQL Database That Scales
- Spanner: Google’s Globally Distributed Database Presentation
- Spanner, TrueTime, and the CAP Theorem
- Spanner: Becoming a SQL System
- Spanner under the hood: Understanding strict serializability and external consistency
- Cloud Spanner 101: Google's mission-critical relational database
- Distributed Systems: Google's Spanner
- Spanner - Google's Distributed Database by Sebastian Kanthak
- Distributed Systems: Two-phase commit
- How Does Google Make Money?
- Block diagrams created with Eraser